

Estructuras de programación

Objetivos

- Comprender la diferencia entre análisis léxico, sintaxis y semántica.
- Identificar los componentes léxicos de Java.
- Construir y evaluar expresiones matemáticas y lógicas.
- Utilizar métodos de la biblioteca estándar de Java.
- Reconocer los distintos tipos de sentencias.
- Entender el concepto de flujo de control.
- Desarrollar algoritmos iterativos.
- Realizar trazas de ejecución.

Contenido

Introducción	3
Análisis léxico y componentes del lenguaje	4
Concepto de Token y separador	4
Palabras reservadas	4
Expresiones y Operadores	4
Operadores	4
Evaluación en cortocircuito	4
Precedencia y asociatividad	4
Conversión de tipos	4
Reglas de promoción	4
Conversión explícita	4
Clases envolventes	4
Uso de la biblioteca estándar	4
La clase Math	4
Instrucciones	4
Sentencias	4
Bloques y ámbito	4
El flujo de ejecución	4
Concepto de flujo de control	4
Estructuras secuenciales	4
Selección	4
Selección simple	4
Selección compuesta	4
Selección múltiple	5
Lógica de anidamiento	5
Repetición	5
Sentencia while	5
Bucles controlados por condición	5
Bucles controlados por contador	5
Diseño de bucles	5
Depuración y verificación	5
Trazas de ejecución	5
Errores comunes	5

Introducción

Hasta este punto de nuestro aprendizaje, hemos comprendido que un programa de ordenador es, en esencia, un algoritmo escrito para ser ejecutado por una máquina. En el tema anterior, exploramos los cimientos: aprendimos a declarar variables, a reconocer tipos de datos primitivos y a utilizar la biblioteca estándar de Java para realizar operaciones de entrada/salida básicas. Sin embargo, si analizamos los programas que hemos construido, observaremos que todos comparten una característica común: son puramente lineales. El ordenador se limita a ejecutar una instrucción tras otra, en el estricto orden físico en que fueron escritas.

No obstante, la programación en el «mundo real» y la resolución de problemas técnicos complejos en entornos productivos requieren una sofisticación mucho mayor. El pensamiento computacional no es una simple sucesión de pasos, sino un tejido o «urdimbre» donde cada hebra lógica debe entrelazarse con precisión. Para lograr esto, un lenguaje de programación debe ser entendido como un conjunto de reglas, símbolos y palabras que nos permiten modelar la realidad.

El proceso por el cual el ordenador interpreta nuestras intenciones comienza con el análisis léxico. Cuando el compilador recibe nuestro código fuente, su primera tarea es identificar tokens, elementos con significado propio que el compilador separa gracias a delimitadores. Como sucede en el lenguaje natural, no podemos formar una oración coherente amontonando palabras, en los lenguajes de programación combinamos estos componentes léxicos para formar expresiones. Algo como $n + 1$, por sí solo, es solo un valor latente; para que sea algo capaz de realizar una tarea, debe integrarse en una estructura que le dé sentido, como una asignación, por ejemplo.

La verdadera potencia del software reside en la capacidad de romper la linealidad del código para establecer un flujo de control. Este define el orden en que las sentencias se ejecutan realmente durante la actividad de la aplicación. Para ilustrarlo, podemos usar la analogía de la conducción: ir por un tramo recto de carretera equivale a una estructura secuencial; sin embargo, al llegar a una bifurcación, debemos tomar una decisión basada en una condición o quizás necesitemos dar varias vueltas hasta encontrar un lugar donde aparcar.

Profundizaremos aquí en cómo construir expresiones y conoceremos la selección y la iteración, herramientas que nos permitirán empezar a desarrollar algoritmos inteligentes, capaces de tomar decisiones autónomas y procesar información de forma eficiente, sentando así las bases necesarias antes de adentrarnos en los fundamentos de la programación orientada a objetos.

Análisis léxico y componentes del lenguaje

Concepto de Token y separador

Palabras reservadas

Expresiones y Operadores

Operadores

Evaluación en cortocircuito

Precedencia y asociatividad

Conversión de tipos

Reglas de promoción

Conversión explícita

Clases envolventes

Uso de la biblioteca estándar

La clase Math

Instrucciones

Sentencias

Bloques y ámbito

El flujo de ejecución

Concepto de flujo de control

Estructuras secuenciales

Selección

Selección simple

Selección compuesta

Selección múltiple

Lógica de anidamiento

Repetición

Sentencia while

Bucles controlados por condición

Bucles controlados por contador

Diseño de bucles

Depuración y verificación

Trazas de ejecución

Errores comunes